TERASWAP

Comprehensive Technical Analysis

Architecture, Security, Operations & Maturity Report

Classification: CONFIDENTIAL

Date: 22 April 2026

Prepared for: TeraHash (Founder / Architect)

Role: Lead Software Architect, Security Reviewer, Technical Product Analyst

Based on full source code review of the TeraSwap repository: 130+ source files, 11 DEX adapters, 20+ API routes, 2 smart contracts, ~323 tests.

1. Executive Technical Summary

1.1 What TeraSwap Is

TeraSwap is a DEX meta-aggregator for Ethereum mainnet (chainId 1). It queries 11 independent decentralised exchange aggregator APIs in parallel (1inch, 0x, Odos, Velora/ParaSwap, KyberSwap, CoW Protocol, Uniswap V3, OpenOcean, SushiSwap, Balancer, Curve), compares their quoted outputs after gas costs, and routes the user's swap through the source offering the best net output. Optionally, a split-routing engine distributes a single swap across multiple sources to maximise output when no single source dominates.

The platform collects a 0.1% fee via a bespoke FeeCollector smart contract that interposes between the user's token approval and the downstream DEX router. An autonomous order engine supports limit orders, stop-loss/take-profit, and dollar-cost averaging (DCA) via a separate OrderExecutor contract with Chainlink oracle triggers and an external keeper network.

The frontend is a Next.js 16 application deployed on Vercel (Hobby tier), with Capacitor wrappers for iOS and Android. State persistence uses Upstash Redis (via REST API) for monitoring/rate-limiting and Supabase (PostgreSQL) for analytics, order storage, and security events.

1.2 How It Works (End-to-End Flow)

The core swap flow proceeds through seven stages: **(1)** the user selects tokens and amount; **(2)** the frontend calls /api/quote, which fans out to all 11 adapters in parallel with 5-second timeouts and circuit-breaker filtering; **(3)** quotes are normalised, gas-adjusted, outlier-filtered (3x median threshold), and sorted; **(4)** the user reviews the best quote via QuoteBreakdown and optionally a split-route visualisation; **(5)** /api/swap fetches execution calldata from the chosen source, validates the calldata recipient against the user's wallet address, checks known swap selectors, and performs a DefiLlama server-side price guard for swaps above $10k; **(6)** the frontend runs an eth_call simulation, displays a TransactionPreview modal with decoded calldata ('clear signing'), and waits for explicit user confirmation; **(7)** the transaction is submitted on-chain, a post-execution validator checks actual vs expected output via Transfer event logs, and any critical shortfall auto-disables the responsible source.

1.3 Maturity Level

TeraSwap is a **late-beta product**. After 8 complete sprints with auditor-verified prompt packets, it has accumulated substantial security infrastructure: constant-time bearer auth, quorum consensus anomaly detection, TLS/DNS fingerprint monitoring, a 4-component monitoring stack with Telegram/Discord/Email alerting, a P0 kill-switch with admin friction, circuit-breaker collapse detection, and a Sybil detector. The architecture is well-documented (5 ADRs, 5 incident reports, 8+ sprint packets, operational hygiene review).

However, the product operates on Vercel Hobby tier (no SLA, no team access controls), has no frontend or hook-level tests (only server-side lib tests), split-swap execution is non-atomic, and the order engine relies on an external keeper network with no fallback. The system is ready for controlled beta with technically sophisticated users, but requires hardening before scaling to a broader audience or handling significant volume.

1.4 Key Strengths

- Defence-in-depth with 7 independent validation layers: input validation, circuit breaker, quorum consensus, calldata recipient verification, known swap selector allowlist, server-side price guard, and post-execution balance check.
- Constant-time SHA-256 bearer token authentication across all admin/monitoring endpoints, preventing timing side-channel attacks.
- Clear signing UX via TransactionPreview that decodes calldata to show recipient, amounts, deadline, and selector validation status before user signs.
- Mature incident response culture: 5 documented incidents with root cause analysis, remediation commits, and auditor verdicts.
- P0 alert classification with prefix-matching, centralised in a shared module after C-01 divergence fix.
- Comprehensive monitoring: Cloudflare Worker cron (60s), GitHub Actions watchdog (5min), TLS/DNS baseline validation, on-chain event scanning.

1.5 Key Weaknesses

- Zero frontend/hook test coverage: SwapBox, useSwap, useApproval, TransactionPreview, TokenSelector — all untested.
- Split-swap non-atomicity: multi-leg swaps execute sequentially; partial failure leaves user with incomplete execution and per-leg MEV exposure.
- KV rate limiter fail-open: if Upstash Redis is unavailable, rate limiting is silently bypassed, exposing all endpoints to abuse.
- Vercel Hobby tier: no SLA, no team access controls, no fine-grained environment scoping, 10s function timeout for most routes.
- In-memory circuit breaker per adapter resets on cold start; takes ~3 failures (~90s) to rebuild protection.
- Systemic circuit breaker (P46) is alert-only, does not automatically halt swap routing when majority of sources fail.

2. System Architecture

2.1 Technology Stack

Layer	Technology	Version / Detail
Frontend Framework	Next.js (App Router)	16.2.3
UI Library	React	18.3.0
Styling	Tailwind CSS	3.4.0
Wallet Integration	RainbowKit + wagmi + viem	2.1.0 / 2.19.5 / 2.47.4
State Management	Zustand + React Query	4.5.0 / 5.50.1
Animations	Framer Motion	12.36.0
Hosting / Deploy	Vercel (Hobby tier)	Next.js runtime
KV Store	Upstash Redis (REST)	@upstash/redis 1.37.0
Database	Supabase (PostgreSQL)	@supabase/supabase-js 2.99.1
Error Monitoring	Sentry	@sentry/nextjs 10.43.0
Mobile	Capacitor (iOS + Android)	8.2.0
Smart Contracts	Solidity (Foundry/Forge)	FeeCollector + OrderExecutor
Monitoring Scheduler	Cloudflare Worker	Cron trigger */60
External Watchdog	GitHub Actions	5-minute polling
Testing	Vitest	4.1.4
Language	TypeScript (strict)	5.5.2

2.2 Module Architecture

The codebase is organised into 7 primary layers:

- **src/app/api/** — 20+ Next.js API routes (server-side, serverless functions). These are the system's external boundary.
- **src/lib/adapters/** — 11 DEX adapter modules, each implementing a normalised interface for fetching quotes and swap calldata.
- **src/lib/** — ~50 modules covering security (auth, rate limiting, circuit breakers, calldata validation), state (KV, Supabase), monitoring (health checks, alerting, on-chain scanning), and business logic (routing, quorum, fee extraction).
- **src/components/** — ~40 React components implementing the swap UI, order panels, analytics dashboard, and security UX (TransactionPreview, Permit2EducationModal, WalletSessionGuard).

- **src/hooks/** — 16 custom hooks orchestrating frontend state, wallet interactions, and data fetching.
- **contracts/** — Solidity smart contracts: TeraSwapFeeCollector (fee interception + router whitelist with 48h timelock) and TeraSwapOrderExecutor (autonomous order execution).
- **workers/** — Cloudflare Worker for monitoring cron trigger.

2.3 Critical Dependencies & Integration Points

Dependency	Purpose	Failure Mode
Upstash Redis (REST)	Rate limiting, monitoring state, circuit breaker, heartbeat, grace periods	Fail-open for rate limiter; P0 alert for state writes; monitoring loop skips tick
Supabase (PostgreSQL)	Analytics, orders, security events, swap/quote logging	Graceful degradation (fire-and-forget); order creation fails
Chainlink Oracles	Price deviation detection, order triggers	Stale price = oracle unavailable; blocks >$10k without alternative
DefiLlama API	Server-side price guard for high-value swaps	Fails-open <$10k; blocks >$10k if unavailable (3s timeout)
11 DEX APIs	Quote/swap data fetching	Circuit breaker per source; quorum consensus detects manipulation
Vercel (Hobby)	Hosting, serverless functions, edge routing	No SLA; 10s timeout; cold starts lose in-memory state
Cloudflare Worker	60s monitoring cron trigger	GitHub Actions watchdog detects staleness at 5-min intervals
Telegram Bot API	Interactive alerts + admin commands	Silent fallback if token/chat missing

2.4 Data Flow: Quote-to-Swap

1. User Input — SwapBox validates token addresses, amounts, balance. Debounced (300ms).

2. /api/quote — Rate-limited by IP. Fans out to 11 adapters via Promise.allSettled (5s timeout each). Circuit breaker filters open sources. Outliers >3x median removed. Gas-adjusted sort. Returns MetaQuoteResult with crossQuoteDeviation warning.

3. Frontend Analysis — useChainlinkPrice checks deviation (warn >=2%, block >=3%). useSplitRoute evaluates multi-leg opportunity (>=10 bps improvement). QuoteBreakdown displays comparison.

4. /api/swap — Rate-limited. Fetches execution calldata from chosen source. Server-side defences: (a) known swap selector validation [SC-04]; (b) recipient-in-calldata verification [R1]; (c) DefiLlama price guard [INT-01] blocks >$10k if oracle unavailable, warns on 2-3% deviation, blocks >3%.

5. Pre-Execution — eth_call simulation catches reverts, insufficient balance, STF errors, slippage failures. TransactionPreview decodes calldata: recipient with badge ('Your wallet' / 'FeeCollector' /

'Unknown'), amounts, deadline, selector status. User must explicitly confirm.

6. On-Chain Execution — sendTransaction (standard) or EIP-712 sign + solver submission (CoW). Fallback receipt polling (3s interval, 2min timeout).

7. Post-Execution — /api/monitor/validate-execution checks Transfer event logs: actual output vs expected. <=2% shortfall = warning; >2% = critical = P0 alert + auto-disable source. KV audit trail (7-day TTL).

2.5 Trust Boundaries

The system operates across three trust domains:

- **Trusted:** On-chain contracts (Uniswap V3, Curve — immutable logic), user wallet (signature authority), Chainlink oracles (with staleness check).
- **Semi-trusted:** DEX aggregator APIs (1inch, 0x, Odos, etc.) — trusted for honest quotes but verified by quorum consensus, calldata recipient validation, and post-execution balance checks.
- **Untrusted:** User input (validated server-side), external HTTP responses (timeout + error handling), browser environment (WalletSessionGuard, Permit2 education).

3. Deep Technical Walkthrough

3.1 Data Entry: Adapter Layer

Each DEX adapter implements a DEXAdapter interface with fetchQuote() and fetchSwapData() methods returning a NormalizedQuote. The 11 adapters fall into 3 categories:

- **API-based (7):** 1inch, 0x, Odos, Velora/ParaSwap, KyberSwap, OpenOcean, SushiSwap. HTTP calls to external APIs with server-only API keys.
- **On-chain (2):** Uniswap V3 (RPC eth_call to QuoterV2), Curve (RPC to CurveRouterNG). No external API dependency.
- **Intent-based (1):** CoW Protocol. Returns order parameters for EIP-712 signing, not transaction calldata. Excluded from split routing.
- **Hybrid (1):** Balancer. SOR API for routing + on-chain vault execution.

All adapters share common utilities: clampSlippage() enforces 0.01-15% range [L-01]; withTimeout() wraps every external call in a 5s Promise race; toWeth() converts native ETH sentinel to WETH; deductFee() extracts the platform fee using integer BPS arithmetic (0.1% = 10 BPS).

3.1.1 Uniswap V3 Fee Tier Auto-Detection

Uniswap V3 pools exist at 4 fee tiers (1, 5, 30, 100 bps). The adapter queries all 4 in parallel via eth_call to QuoterV2.quoteExactInputSingle(). Selection: highest amountOut wins; on tie, lowest gasEstimate wins. Results are cached for 45 minutes per token pair (in-memory TTL cache). This is the most sophisticated adapter and avoids relying on external APIs entirely.

3.1.2 CoW Protocol Special Path

CoW is fundamentally different: it uses off-chain signed orders submitted to a solver network for batch auction settlement. The adapter returns cowOrderParams instead of a transaction object. The frontend detects this and switches to an EIP-712 signing flow. Orders settle within 30-second batch windows. CoW is excluded from split routing because intent-based orders cannot be sub-divided.

3.2 Processing: Meta-Quote Orchestration

The core orchestrator (api.ts) implements fetchMetaQuote(), which fans out to all non-disabled, circuit-breaker-closed sources via Promise.allSettled(). Failed or timed-out sources are silently excluded. Valid quotes (toAmount > 0) are sorted by net output after gas costs. An outlier filter removes any quote exceeding 3x the median. A crossQuoteDeviation flag warns if the best quote deviates >5% from the median.

A global rate limiter (30 req/min) throttles outbound API calls. A hardcoded DISABLED_SOURCES map permanently disables sources pending reactivation review (e.g., CoW Protocol post-2026-04-14 DNS hijack).

3.3 Validation Stack (7 Layers)

Layer 1: Input Validation

Centralised in validation.ts: regex-based address validation (0x + 40 hex), transaction hash validation (0x + 64 hex), amount validation (positive finite number). safeCompare() implements constant-time comparison via SHA-256 hashing + timingSafeEqual() for bearer tokens [API-CRITICAL-01].

Layer 2: Circuit Breaker (Per-Adapter)

In-memory state machine per source: CLOSED -> OPEN (3 consecutive failures) -> HALF_OPEN (60s cooldown, 1 test request) -> CLOSED (on success). Prevents hammering failing APIs. Resets on Vercel cold start; rebuilds in ~3 failures (~90s).

Layer 3: Quorum Consensus

Every 5th monitoring tick (~5 min), runQuorumCheck() fetches quotes for 2 reference pairs (1 ETH -> USDC at 5% threshold; 10k USDC -> USDT at 2% threshold) from all active sources. IQR pre-filter removes statistical outliers. BigInt-safe deviation computation: |sourceAmount - median| x 10000 / median. Sources deviating on both pairs are disabled. If >=3 sources flag simultaneously, a correlated anomaly triggers P0 kill-switch requiring manual recovery. Minimum 5 active sources required for quorum validity.

Layer 4: Calldata Recipient Validation [R1]

calldata-recipient.ts extracts the recipient address from swap calldata by recognizing 19 known function selectors across 6 groups: Group A (msg.sender implicit), Groups B-D (recipient as explicit parameter), Group E (multicall wrappers with recursive validation), Group F (trusted routers). FAIL-CLOSED: unknown selectors are blocked by default [API-M-02]. Valid recipients: user wallet OR FeeCollector address.

Layer 5: Known Swap Selector Allowlist [SC-04]

swap-selectors.ts maintains a whitelist of 19 known DEX function selectors. getSelector() extracts the 4-byte function selector. isKnownSwapSelector() enforces the allowlist. Used both client-side (pre-validation) and server-side (defence-in-depth).

Layer 6: Server-Side Price Guard [INT-01]

DefiLlama provides a free secondary oracle. For swaps >$10k USD, the server blocks execution if the oracle is unavailable (3s timeout, 2-min TTL cache). For 2-3% deviation: warning logged. For >=3% deviation: swap blocked with HTTP 422 and priceGuard flag. Fails-open for <$10k.

Layer 7: Post-Execution Balance Check [P45]

After on-chain execution, the validator fetches the TX receipt, extracts Transfer event logs (topic[0] = keccak256 of Transfer, topic[2] = recipient), and sums all matching transfers for tokenOut. Fallback:

balanceOf diff if preSwapBalance was provided. Shortfall classification: ok (>=0), warning (0-2%), critical (>2% -> P0 alert + source auto-disable). KV audit trail with 7-day TTL.

3.4 Persistence & State

State is distributed across 4 backends:

- **Upstash Redis (REST):** Monitoring state (source health, heartbeat, grace period, quorum results, circuit breaker trips), rate limiter counters (sliding window sorted sets), alert dedup counters, on-chain scan block cursor.
- **Supabase (PostgreSQL):** Swap logs, quote logs, security events, wallet activity, orders, execution history. Service-role key (no RLS) for server-side; anon key (RLS enforced) for client-side order operations.
- **localStorage:** Client-side analytics events, source preferences, Permit2 education state, session ID, swap history.
- **In-memory:** Per-adapter circuit breaker state, source health metrics (500-ping rolling buffer), outbound rate limiter, fee tier cache.

3.5 Error Handling & Fallback

The system follows a consistent philosophy: critical paths fail-closed, analytics paths fail-open.

- **Adapter failures:** Circuit breaker opens after 3 failures; source state machine transitions to degraded/disabled; quorum consensus runs independently.
- **KV failures:** Rate limiter fails-open (allows requests); state machine writes emit P0 alert; monitoring loop skips tick and retries next cycle.
- **Supabase failures:** All logging is fire-and-forget (Promise.resolve().catch()); never blocks swap flow.
- **Price oracle failures:** DefiLlama fails-open <$10k, blocks >$10k. Chainlink stale = oracle unavailable (blocks at danger threshold).
- **RPC failures:** Post-execution validator returns severity 'unknown' (non-blocking). Receipt polling has 2-min timeout with fallback.
- **Alert channel failures:** Each channel (Telegram/Email/Discord) fails independently via Promise.allSettled(); never cascades.

4. Security Review

Findings organised by severity. Each finding includes technical explanation, real-world impact, risk assessment, affected area, and recommendation.

4.1 Critical Findings

No unresolved critical findings. Previous criticals (C-01: P0 detection divergence via exact string match instead of prefix match) have been resolved with the shared p0-reasons.ts module. The Vercel breach (INC-2026-04-19-001) has been fully remediated with secret rotation.

4.2 High Findings

[High] H-01: KV Rate Limiter Fail-Open Design

Explanation: kv-rate-limiter.ts returns { allowed: true } when Upstash Redis is unavailable. This means all rate limiting (quote: 30/min, swap: 20/min, RPC: 60/min) is silently bypassed during any KV outage.

Impact: An attacker monitoring for KV downtime could flood all endpoints without restriction. The /api/rpc endpoint becomes an open proxy for 12 whitelisted Ethereum RPC methods.

Real Risk: MEDIUM-HIGH. Upstash has good availability, but KV outages have occurred (INC-2026-04-14-002 shows 13-day silent misconfiguration). The fail-open design prioritises availability over security.

Area: *src/lib/kv-rate-limiter.ts*

Recommendation: Implement an in-memory fallback rate limiter that activates when KV is unavailable. Use a conservative limit (e.g., 50% of normal). Log KV fallback activation as a warning event.

[High] H-02: Split Swap Non-Atomicity and MEV Exposure

Explanation: useSplitSwap.ts executes multi-leg swaps sequentially (one transaction per leg). If leg 2 of 3 fails, the user has already executed leg 1 with partial output. Each leg is an independent on-chain transaction visible to MEV searchers.

Impact: Partial execution leaves the user with an incomplete swap and potentially unfavourable intermediate token positions. Each leg is independently sandwichable by MEV bots.

Real Risk: MEDIUM. Split routing is optional and requires user opt-in. However, users may not understand the atomicity implications. The improvement threshold (10 bps) may not justify the MEV risk.

Area: *src/hooks/useSplitSwap.ts, src/lib/split-router.ts*

Recommendation: Add explicit warning in SplitRouteVisualizer about non-atomicity and MEV risk. Consider raising the improvement threshold to 50+ bps. Explore multicall-based atomic splits via a custom router.

[High] H-03: Systemic Circuit Breaker Is Alert-Only

Explanation: circuit-breaker.ts (P46) detects systemic collapse (>=6 sources disabled or >=4 in 10-min cascade) but only emits a P0 alert. It does not automatically pause swap routing.

Impact: During a coordinated attack disabling multiple sources, the system continues operating with reduced coverage. If the remaining sources are being manipulated, users are routed directly to the attacker.

Real Risk: MEDIUM. With fewer than 5 sources the quorum check is skipped. This creates a gap: systemic collapse + quorum skip = no manipulation detection.

Area: *src/lib/circuit-breaker.ts, src/lib/monitoring-loop.ts*

Recommendation: When the systemic circuit breaker trips, automatically set a global KV flag that /api/quote and /api/swap check. If set, return HTTP 503 with a maintenance message. Require manual clearance.

[High] H-04: FeeCollector Forwards Arbitrary Calldata Without Output Validation

Explanation: TeraSwapFeeCollector.sol pulls user tokens, deducts fee, approves the router, and forwards arbitrary calldata. The contract does not validate that the router returned expected output.

Impact: If a whitelisted router is compromised or returns less than expected, the FeeCollector cannot prevent the loss. Post-execution validation is asynchronous and can only detect, not prevent.

Real Risk: MEDIUM. The router whitelist + 48h timelock significantly reduces risk. However, routers are upgradeable proxies (1inch, 0x), so a compromised upgrade could affect all users until timelock removal.

Area: *contracts/TeraSwapFeeCollector.sol*

Recommendation: Add a minimumOutput parameter to swapTokenWithFee() and validate the user's tokenOut balance after router execution. Revert if output < minimumOutput.

4.3 Medium Findings

[Medium] M-01: No Frontend or Hook Test Coverage

Explanation: Zero automated tests exist for any React component or custom hook. SwapBox, TransactionPreview, Permit2EducationModal, useSwap, useApproval, useQuote — all untested.

Impact: Regressions in the swap flow, approval logic, calldata display, or price guard UI enforcement could ship to production without detection.

Real Risk: HIGH. The frontend is the user's last line of defence before signing. Untested security UX is a liability.

Area: *src/components/, src/hooks/*

Recommendation: Prioritise integration tests for: (1) useSwap fee validation; (2) TransactionPreview calldata decoding; (3) SwapBox Chainlink price guard; (4) Permit2EducationModal display trigger.

[Medium] M-02: In-Memory Adapter Circuit Breaker Loses State on Cold Start

Explanation: Per-adapter circuit breaker uses in-memory Map. Every Vercel cold start resets all breakers to CLOSED. Takes 3 failures (~90s) to re-open for a persistently failing source.

Impact: During the 90-second rebuild window, requests are sent to sources that were previously failing.

Real Risk: LOW-MEDIUM. The KV source state machine provides secondary protection, but the two layers are not synchronised.

Area: *src/lib/adapters/circuit-breaker.ts*

Recommendation: On adapter init, check KV source state. If disabled/degraded in KV, pre-set adapter circuit breaker to OPEN.

[Medium] M-03: Supabase Service-Role Key Grants Full Write Access

Explanation: supabase.ts uses the service-role key (bypasses RLS) for all server-side operations. This key can read, write, and delete any row in any table.

Impact: If leaked, an attacker has full database access. Fire-and-forget patterns mask modifications.

Real Risk: MEDIUM. The key is server-only. However, INC-2026-04-19-001 showed env var exposure is possible.

Area: *src/lib/supabase.ts*

Recommendation: Create a dedicated Supabase role with INSERT-only permissions for logging tables. Use service-role key only for order operations.

[Medium] M-04: Grace Period Alert Suppression Inconsistent Across Channels

Explanation: alert-wrapper.ts sends grace-period alerts only to Telegram (with [GRACE] tag). Email and Discord receive full alerts during grace periods.

Impact: Email/Discord recipients receive alert floods during planned maintenance while Telegram sees grace-tagged alerts. Creates confusion and alert fatigue.

Real Risk: LOW. Functional but inconsistent.

Area: *src/lib/alert-wrapper.ts*

Recommendation: Apply the [GRACE] tag to all channels during grace periods, or suppress non-P0 alerts entirely with a summary at grace end.

[Medium] M-05: On-Chain Monitor 5-Minute Cadence for Critical Events

Explanation: on-chain-monitor.ts scans every 5th tick (~5 min). Critical events (AdminTransferred, Paused, OwnershipTransferred) are detected with up to 5+ minutes of delay.

Impact: Malicious admin actions would not be detected for minutes, during which users may interact with compromised contracts.

Real Risk: LOW-MEDIUM. The 48h timelock provides a larger window. However, Paused/Unpaused bypass timelock.

Area: *src/lib/on-chain-monitor.ts*

Recommendation: Run on-chain scanning every tick (60s). For critical events, consider WebSocket subscription.

4.4 Low Findings

[Low] L-01: CORS Wildcard on Logging Endpoints

Explanation: log-activity and log-event set Access-Control-Allow-Origin: *. Any origin can submit events.

Impact: Attacker could flood logging endpoints with fake analytics data.

Real Risk: LOW. Analytics-only data, does not affect swap execution.

Area: *src/app/api/log-activity/route.ts, src/app/api/log-event/route.ts*

Recommendation: Restrict CORS to teraswap.app and localhost origins.

[Low] L-02: localStorage Quota Risk for Analytics

Explanation: analytics.ts stores trade events in localStorage. JSON parsing has no validation. Quota is 5-10MB.

Impact: Power users may hit quota, causing silent write failures. Corrupted JSON could crash dashboard.

Real Risk: LOW. Affects analytics display only.

Area: *src/lib/analytics.ts*

Recommendation: Add try-catch around JSON.parse with reset fallback. Implement rolling window (last 1000 events).

[Low] L-03: Telegram Inline Callback Buttons Not User-Validated

Explanation: Callback queries (activate, keep, escalate, ack) do not verify the button-clicker is in TELEGRAM_ADMIN_IDS. Any group member can click action buttons.

Impact: Non-admin users could activate/disable sources via inline buttons.

Real Risk: LOW-MEDIUM. Only if the Telegram group has non-admin members.

Area: *src/app/api/telegram/webhook/route.ts*

Recommendation: Add TELEGRAM_ADMIN_IDS check to callback query handler.

[Low] L-04: Seed Data Guard Only Checked at Runtime

Explanation: analytics.ts includes 350-trade demo dataset with runtime NODE_ENV guard.

Impact: Misconfigured NODE_ENV could pollute production analytics.

Real Risk: VERY LOW. Guard is simple and well-documented.

Area: *src/lib/analytics.ts*

Recommendation: Move seed data to dev-only module excluded from production build.

5. Operational Review

5.1 Monitoring Architecture

The monitoring stack is a 4-component system designed for zero operational cost:

- **H1 (Health Check):** Every tick probes teraswap.app (self) and all aggregator APIs with GET/HEAD requests (8s timeout). Results feed the source state machine.
- **H2 (TLS/DNS Fingerprint):** Validates TLS certificate issuer and DNS A/NS records against a committed baseline. Detects DNS hijacking and certificate substitution. P0 on any mismatch.
- **H5 (Quorum Consensus):** Every 5th tick, cross-checks reference pair quotes across active sources. IQR-filtered, BigInt-safe deviation. Correlated outliers trigger P0.
- **H6 (Telegram Bot):** Interactive bot with /status, /quorum, /heartbeat. Admin commands: /disable, /lock (P0), /activate, /grace. Inline buttons on alerts.

The monitoring loop (monitoring-loop.ts) is called by a Cloudflare Worker every 60 seconds via POST /api/monitor/tick. It acquires a KV-based distributed lock (55s TTL), runs all checks sequentially, writes a heartbeat, and releases the lock. A GitHub Actions watchdog polls GET /api/monitor/heartbeat every 5 minutes and escalates if healthy !== true.

5.2 Alert Routing & Deduplication

Alerts route through alert-wrapper.ts with fan-out to Telegram, Email (Resend), and Discord (webhook). Deduplication uses KV counters: standard alerts get 15-min TTL with max 3 per window; P0 alerts get 5-min TTL with unlimited delivery. Oscillation detection on 3rd occurrence suggests grace duration.

5.3 Fragility Points

- **Single KV dependency:** All monitoring state, rate limiting, circuit breaker state, and alert dedup use a single Upstash Redis instance. KV failure degrades multiple systems simultaneously.
- **Vercel cold starts:** In-memory state (adapter circuit breakers, source metrics buffer, outbound rate limiter) is lost. Warmup detection skips latency recording, but circuit breakers reset to CLOSED.
- **GitHub Actions watchdog 5-min resolution:** If Cloudflare Worker and Vercel both fail, detection takes up to 5 minutes. Not suitable for time-critical DeFi incidents.
- **No distributed transaction guarantees:** The monitoring loop performs multiple KV writes sequentially. A crash mid-tick can leave partial state.
- **eth_getLogs replay risk:** On-chain monitor failures don't advance the block cursor, potentially replaying events on next tick.

5.4 Observability

Sentry captures client-side and server-side errors with intelligent filtering. Trace sampling at 10%. Server-side security events logged to Supabase. Monitoring state queryable via /api/monitor/status (public) and /api/monitor/heartbeat/admin (authenticated).

Gap: No structured logging framework (Pino, Winston). console.log/warn/error is the primary mechanism. Vercel Hobby tier provides minimal log retention.

5.5 Test Coverage

Module Category	Test Files	Assessment
Auth & Security	auth, circuit-breaker, source-thresholds	Good: timing-safe, state transitions, P0 blocking
Calldata Validation	calldata-decoder, calldata-recipient	Good: selector groups, recipient extraction, unknown blocking
Post-Execution	post-execution-validator	Good: Transfer parsing, shortfall classification
Monitoring	monitoring-loop, on-chain-monitor	Good: tick orchestration, event severity
Alerting	alert-wrapper, telegram	Good: fan-out, dedup, grace, callbacks
API Routes	kill-switch, heartbeat, status, tick, validate-execution, telegram/webhook	Good: auth, validation, error responses
Frontend Components	NONE	CRITICAL GAP: zero coverage on 40+ components
Hooks	NONE	CRITICAL GAP: zero coverage on 16 hooks
Integration / E2E	NONE	GAP: no end-to-end swap flow tests

6. Product Maturity Assessment

6.1 What Is Solid

- **Server-side security infrastructure:** Auth, rate limiting, calldata validation, quorum consensus, and post-execution validation are well-designed, well-tested, and follow defence-in-depth principles.
- **Monitoring and incident response:** 4-component stack with zero cost. 5 documented incidents with root cause analysis. P0/non-P0 classification with prefix-matching is mature.
- **ADR-driven architecture:** 5 formal decision records with alternatives considered, cost analysis, and rationale. Uncommon for a project at this stage.
- **Security UX:** TransactionPreview (clear signing), Permit2EducationModal (anti-phishing), WalletSessionGuard (idle timeout), ActiveApprovals (revocation), TokenAddressBadge (verification).
- **Smart contract design:** FeeCollector with ReentrancyGuard, router whitelist, 48h timelock, pause mechanism, and approval revocation after execution.

6.2 What Is Incomplete

- **Frontend testing:** Zero component and hook tests. The security UX layer that protects users from signing malicious transactions is entirely untested.
- **Order engine:** Functional but beta. No keeper network integrated; execution relies on an unverified external party.
- **Multi-chain support:** Hardcoded to chainId 1. No abstraction layer for multi-chain expansion.
- **User authentication:** No account system. All operations are wallet-based. No cross-device history.
- **Analytics dashboard:** localStorage as source of truth. No server-side analytics API. Sybil flags not propagated to backend.

6.3 What Is Fragile

- **KV single point of failure:** Upstash Redis is the backbone for rate limiting, monitoring, alert dedup, circuit breaker, and grace periods. A KV outage degrades 5+ systems simultaneously.
- **Vercel Hobby tier:** No SLA, 10s function timeout, no team access controls, minimal log retention.
- **In-memory state on serverless:** Adapter circuit breakers, source metrics, outbound rate limiter, and fee cache all reset on cold start.
- **Split routing execution:** Non-atomic multi-leg swaps with per-leg MEV exposure.
- **Grace period dual-source:** Env var + KV dual-source creates edge cases where one expires while the other is active.

6.4 What Needs Hardening Before Scaling

- Upgrade to Vercel Pro (or equivalent) for SLA guarantees, extended timeouts, team access controls, and log retention.
- Add frontend integration tests for critical security paths.
- Implement KV fallback for rate limiting.
- Add minimumOutput validation to FeeCollector contract.
- Convert systemic circuit breaker from alert-only to alert-and-halt.
- Reduce on-chain scan cadence from 5-minute to 60-second.
- Implement structured logging with external aggregation.

7. Priority Recommendations

Ordered by risk reduction per unit of effort. Items marked with severity from the Security Review where applicable.

Priority	Action	Rationale	Effort
P0	Add minimumOutput to FeeCollector [H-04]	On-chain protection against compromised routers	1-2 days
P0	Convert circuit breaker to alert-and-halt [H-03]	Prevents user exposure during systemic collapse	0.5 day
P1	Add frontend integration tests [M-01]	Zero coverage on user-facing security UX	3-5 days
P1	Implement KV rate limiter fallback [H-01]	In-memory backup prevents full bypass	1 day
P1	Validate Telegram callback senders [L-03]	Prevents non-admin source control	0.5 day
P2	Sync adapter circuit breaker with KV state [M-02]	Eliminates 90s cold-start vulnerability	1 day
P2	Reduce on-chain scan cadence to 60s [M-05]	Near-real-time critical event detection	0.5 day
P2	Fix grace period alert suppression [M-04]	Consistent [GRACE] tagging across channels	0.5 day
P2	Add split-swap MEV warning [H-02]	User must acknowledge non-atomicity risk	0.5 day
P3	Restrict CORS on logging endpoints [L-01]	Prevents cross-origin analytics pollution	0.5 day
P3	Separate Supabase roles [M-03]	Limits blast radius of key compromise	1 day
P3	Implement structured logging	Replaces console.log; enables audit trails	2-3 days
P3	Move seed data to dev-only module [L-04]	Build-time exclusion vs runtime guard	0.5 day
P4	Upgrade Vercel to Pro tier	SLA, team access, extended timeouts	Operational
P4	Add localStorage quota management [L-02]	Rolling window + corrupted JSON recovery	0.5 day

7.1 Architecture Decisions to Revisit

- **Split routing atomicity:** Evaluate a custom multicall router contract that executes all legs atomically. Eliminates per-leg MEV exposure and partial failure states.
- **In-memory vs persistent circuit breakers:** The dual-layer approach creates synchronisation complexity. Consider a single KV-backed circuit breaker with local caching.
- **Fire-and-forget logging:** While acceptable for analytics, consider synchronous logging for security events to ensure forensic completeness.
- **Multi-chain architecture:** Before adding chains, extract chain-specific constants into a chain configuration module. The current hardcoded approach will not scale.
- **Order engine keeper integration:** Define SLA requirements for the keeper network. Without a fallback executor, user orders may expire during keeper downtime.

— End of Report —